

CSCI 201 Team 8 Final Project: College Football Stats Tracker

Documentation written by:

Jushen Dhillon
Adl Gamaladdin
Ernest Guo
Justin Jiang
Amit Krishna
Nguyen Nguyen
Marina White

Table of Contents

Concept.....	3
High Level Requirements.....	4
Technical Specifications.....	5
Detailed Design.....	8
Testing.....	21
Deployment.....	42

Concept

Our project is a College Football Stat Tracker that retrieves and visualizes real-time NCAA football data using public APIs like *CollegeFootballData.com*. In addition to displaying team and player stats, it includes a projection feature that uses historical performance data and recent trends to predict upcoming game outcomes and player performances through an interactive Java-based dashboard. Our target audience consists of college football viewers, sports betters, and football teams.

High Level Requirements

The College Football Stat Tracker will allow users to view and explore real-time NCAA football data through an easy-to-use interactive dashboard web interface. Users will be able to select teams, players, and seasons they want to track, and the system will display live statistics, historical data, and game results in clear charts and tables. The program will also provide performance projections based on recent trends and past results (through APIs) to predict upcoming games and player outcomes. The tool is designed for college football fans, sports bettors, and team analysts who want quick access to live stats and performance insights without needing technical knowledge.

Technical Specifications

I. Authentication and User Management (12 hours)

- Implement Google Single Sign-On (SSO) for user authentication, supporting both guest and registered users.
 - Provide role-based access (e.g., regular users, premium users, admins) to customize feature availability.
 - Secure session management to protect user accounts and session data.
 - Logout and session timeout functions.
-

II. System Architecture and Database Design (16 hours)

- Use a relational database (MySQL) to store user profiles, teams, player stats, and game results.
 - Suggested schema:
 - Users: userid, email, role, created_at
 - Teams: teamid, name, conference, logo_url
 - Players: playerid, name, teamid, position, stats
 - Games: gameid, date, home_team, away_team, scores, key statistics
 - Stats: statid, playerid, gameid, season, type (e.g., yards, TDs), value
 - Employ prepared statements and input validation for security.
 - SSL-encrypted (Secure Sockets Layer) database connections.
-

III. Data Ingestion and API Integration (10 hours)

- Integrate with third-party sports data APIs to fetch live NCAA football stats and historical records.
 - Schedule regular syncing and updates (every minute for live games).
 - Parse, validate, and store incoming API data to the database.
 - Implement robust error handling and logging for data integrity.
-

IV. Interactive Dashboard and User Interface (20 hours)

- Develop a responsive web interface using modern frameworks (React/Vue.js/Angular).
 - Core features:
 - Team and player selection panels
 - Live statistics visualization (tables, bar/line charts)
 - Historical stats browsing with filters by team/player/season
 - Game result displays
 - Performance projections and predictions
 - Design for accessibility and clarity, enabling use on desktop and mobile devices.
-

- V. Performance Projection Module (16 hours)
 - Implement statistical models (e.g., linear regression, moving averages) using Python or built-in JavaScript libraries to generate projections for upcoming games and players.
 - Visualize predicted outcomes in the dashboard.
 - Provide metrics for model accuracy and display confidence intervals.
-

VI. Multi-threading and Networking (8 hours)

- Backend uses asynchronous data fetching and multi-threaded processing for live stats updates.
 - Network requests are managed via API endpoints for robustness and speed.
-

VII. User Login Functionality (4 hours)

- Differentiate feature access for guests vs. authenticated users (e.g., save preferences, advanced analytics).
 - Public dashboard available for all users, with enhanced features for logged-in accounts.
 - Guest vs User
 - Guest
 - "Publicly accessible subset — delayed and limited depth"
 - realtime_delay_minutes: 10
 - seasons_available: 1
 - teams_visible: "Top 25 only"
 - No projections enabled
 - No Odds data
 - User
 - - realtime_delay_minutes: 0
 - seasons_available: "all"
 - teams_visible: "all"
 - stats_depth: "full (includes advanced metrics)"
 - Projections Data
 - Odds Data
-

VIII. Testing, Deployment, and Maintenance (16 hours)

- Version control managed with GitHub (CI/CD optional for full points).
 - Implement unit, integration, and system-level tests (using Jest, PyTest, etc.).
 - Deploy on a cloud platform (AWS/GCP/Azure); ensure routine backups.
-

IX. Data Structures (4 hours)

- Store team/player/game info in normalized database tables.

- Use in-memory data structures (e.g., HashMaps for quick lookups, Queues for update scheduling) within backend services.
-

X. Skills and Tools

- Web frameworks (React/Vue.js/Angular), RESTful API integration, Python/R for data modeling, MySQL for backend storage.
- Version control: Git/GitHub.
- Data visualization: D3.js, Chart.js, or equivalent.

Detailed Design

Overview

The College Football Stat Tracker is a web-based platform designed to provide users with real-time and historical NCAA football data through a dashboard interface. The system aggregates live statistics from sports data APIs, stores them in a secure relational database, and visualizes them with interactive charts and tables.

Users can log in with Google Single Sign-On (SSO) to unlock advanced analytics features such as projections, odds data, and personalized dashboards. Guest users can browse limited public data with delayed updates.

The system integrates Python (for data ingestion and statistical modeling), Java (for backend APIs), MySQL (for persistent storage), and React.js (for the front end).

Authentication & Security

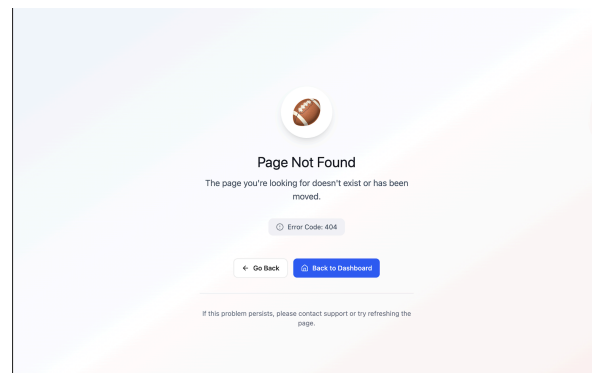
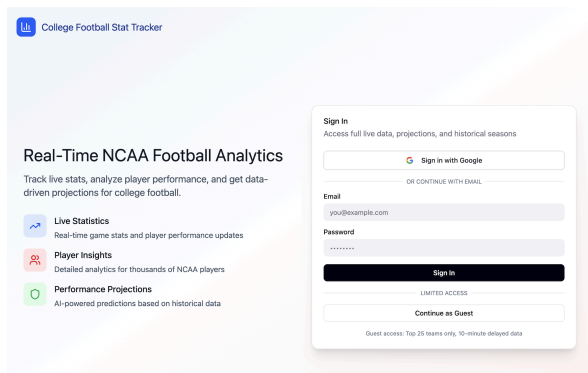
Auth0 Integration (Google SSO):

1. User clicks “**Login**” in the web interface.
2. Redirected to Auth0-hosted page supporting Google (USC SSO) or email/password login.
3. Auth0 returns a secure JSON Web Token (JWT) to the client upon successful authentication.
4. Backend validates JWT with Auth0 public key before granting access to protected routes.

Roles and Permissions:

Guest: Can only view data with a 10-minute delay and only limited to the Top 25 teams.

User: Access to full live data, projections, and historical seasons.



Mockup:

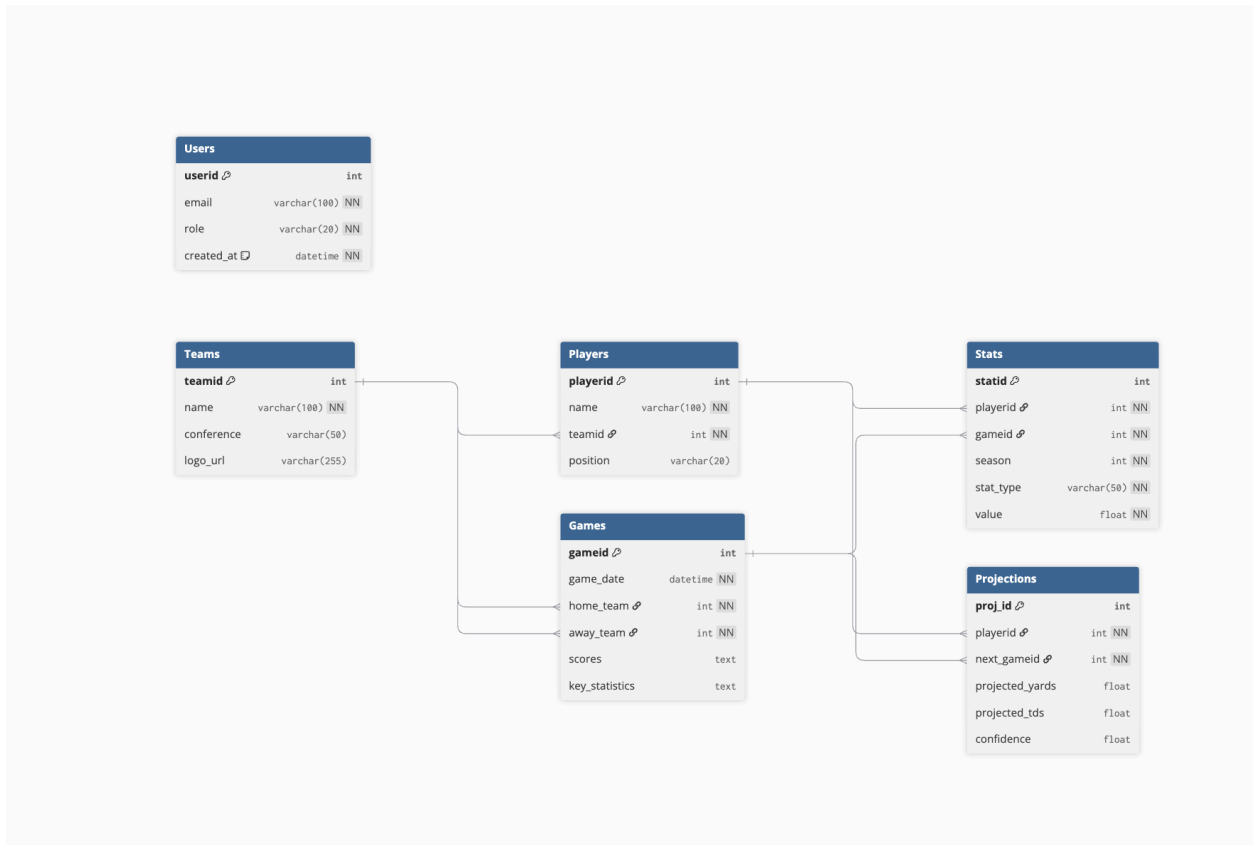
Error Page(s):

Database

MySQL

-
- **Tables**
 - **Users**
 - userid (INT, primary key, auto increment)
 - email (VARCHAR(100))
 - role (VARCHAR(20)) — e.g. guest, user, admin
 - created_at (DATETIME)
 - **Teams**
 - teamid (INT, primary key, auto increment)
 - name (VARCHAR(100)) — team name
 - conference (VARCHAR(50))
 - logo_url (VARCHAR(255))
 - **Players**
 - playerid (INT, primary key, auto increment)
 - name (VARCHAR(100))
 - teamid (INT, FK → Teams.teamid)
 - position (VARCHAR(20))
 - **Games**
 - gameid (INT, primary key, auto increment)
 - game_date (DATETIME)
 - home_team (INT, FK → Teams.teamid)
 - away_team (INT, FK → Teams.teamid)
 - scores (TEXT) — can store JSON-like score data

- key_statistics (TEXT)
- **Stats** (player stats per game)
 - statid (INT, primary key, auto increment)
 - playerid (INT, FK → Players.playerid)
 - gameid (INT, FK → Games.gameid)
 - season (INT)
 - stat_type (VARCHAR(50)) — e.g. passing_yards, rushing_yards, tds
 - value (FLOAT)
- **Projections** (generated by your performance module)
 - proj_id (INT, primary key, auto increment)
 - playerid (INT, FK → Players.playerid)
 - next_gameid (INT, FK → Games.gameid)
 - projected_yards (FLOAT)
 - projected_tds (FLOAT)
 - confidence (FLOAT)
- **API_Log** (*optional but useful for debugging ingestion*)
 - log_id (INT, primary key, auto increment)
 - source (VARCHAR(50)) — which API endpoint
 - fetched_at (DATETIME)
 - status (VARCHAR(20)) — success / failed
 - message (TEXT)



Class Diagram:

BaseService

|

└─ UserService

| └─ loginWithGoogle()

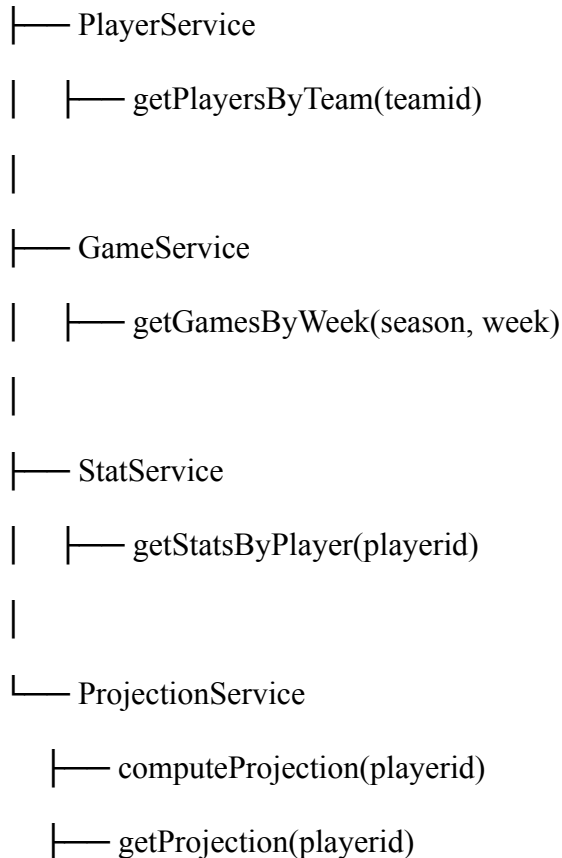
| └─ getUserRole()

|

└─ TeamService

| └─ getTeams()

|



Data Collection:

The data collection system is an essential internal service used to automate the process of updating our MySQL database with the latest college football information (teams, games, players, and player/game stats). Without this system, the application would have to call the third-party sports API every time a user loads the dashboard, which would be slower, more expensive (API rate limits), and harder to keep consistent across users. By ingesting the data first and storing it in our own database, the web app can serve fast, pre-formatted data to both guest and logged-in users.

This program will be written in Python and will rely on a college football / sports data API wrapper to collect live and historical NCAA football data. The ingestion service will be able to run on a schedule (e.g., every minute during live games, every few hours in off-season) or be run manually by an admin.

Main goals:

1. Pull fresh data from the external API.

2. Normalize it to match our database schema (Users, Teams, Players, Games, Stats, Projections).
3. Upsert (insert or update) into MySQL.
4. Log any failures for debugging.

Some Functions needed:

- `get_all_teams(season: int)`
 - Fetch all FBS/FCS teams for a given season from the external football API.
 - Normalize field names (e.g., “school” → name, “conference” → conference).
 - Returns a List[Dict] of team objects.
- `get_games_for_week(season: int, week: int)`
 - Fetch scheduled or completed games for a specific week.
 - Used heavily during the regular season.
 - Returns a List[Dict] where each dict contains: date, home team, away team, score, and basic stats.
- `get_player_stats(game_id: str)`
 - Fetch player-level stats for a specific game.
 - Maps directly to our Stats table (playerid, gameid, stat_type, value).
 - Returns a List[Dict].
- `sync_to_mysql(data: dict, table_name: str)`
 - Generic helper that takes normalized data and either INSERTs or UPDATEs rows in MySQL.
 - Uses prepared statements to avoid SQL injection.
 - This is the “bridge” between the API layer and our database schema.
- `run_ingestion(season: int, week: int)` (*public entry point function*)
 - This is the main function the cron job or admin runs.
 - Steps:
 1. `teams = get_all_teams(season)`
 2. Insert/update Teams table.
 3. `games = get_games_for_week(season, week)`
 4. Insert/update Games table.
 5. For each game, call `get_player_stats(game_id)` and insert into Stats.
 - Returns a summary (how many teams/games/stats were updated).

Data Stored in SQL

- Teams Table
 - teamid (primary key)
 - name
 - conference

- logo_url
- Players Table
 - playerid (primary key)
 - name
 - teamid
 - position
- Games Table
 - gameid (primary key)
 - game_date
 - home_team
 - away_team
 - scores
 - key_statistics
- Stats Table
 - statid (primary key)
 - playerid
 - gameid
 - season
 - stat_type
 - value

Because this system writes straight into these tables, the front end never has to guess the shape of the data, it will already be in the exact format our dashboard expects.

Web Interface:

The College Football Stat Tracker web interface is a responsive dashboard-style application built with React.js, designed to present NCAA football data through interactive charts, tables, and detail views. Guest and authenticated users see the same core layout but with different levels of data access (e.g., live vs. delayed data, Top 25 vs. all teams).

Global Layout & Navigation

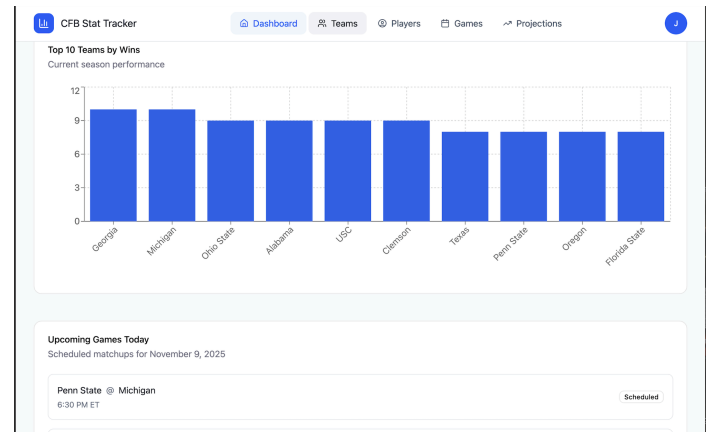
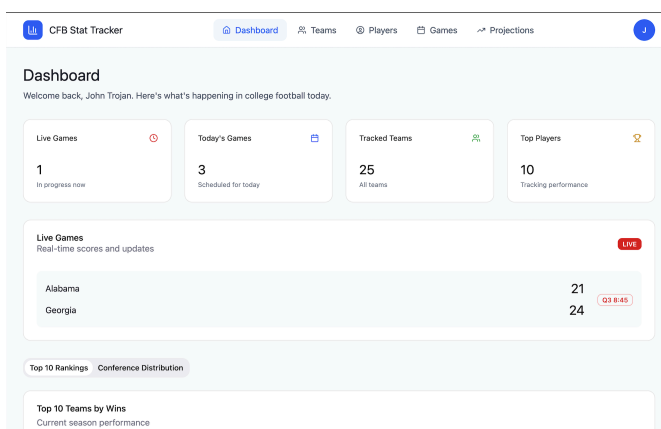
- Two-panel layout (desktop):
 - The right ~4/5 of the screen is dedicated to the main content area (dashboard charts, tables, game and player details).
 - The left ~1/5 of the screen serves as a vertical control panel with global filters (season, week, conference, team), quick stats summaries, and navigation shortcuts (Dashboard, Teams, Players, Games, Projections). This mirrors the split layout used in the Mo example, where the primary visualization sits beside a structured control/information panel.
- Top navigation bar:
 - App logo/title ("College Football Stat Tracker").
 - Links or tabs for the main views: Dashboard, Teams, Players, Games, Projections.

- Authentication controls (Login / Logout, current user email or avatar).
- Role-based UI:
 - Guest users see a banner indicating that data is delayed by 10 minutes and limited to Top 25 teams, with disabled or “locked” badges on advanced analytics sections.
 - Logged-in users (via Google SSO / Auth0) see “Live” badges, access to all teams, full projections, and can toggle advanced filters.
- Visual design:
 - Color accents based on team colors and conference tags (using logo and conference data from the Teams table), with neutral backgrounds for the main dashboard.
 - Consistent typography (e.g., bold sans-serif for headings, regular sans-serif for table/body text), similar in spirit to the clear, modern schedule layout from the Mo example.

Dashboard view:

The Dashboard is the default landing view after login and serves as the high-level overview of current and historical activity.

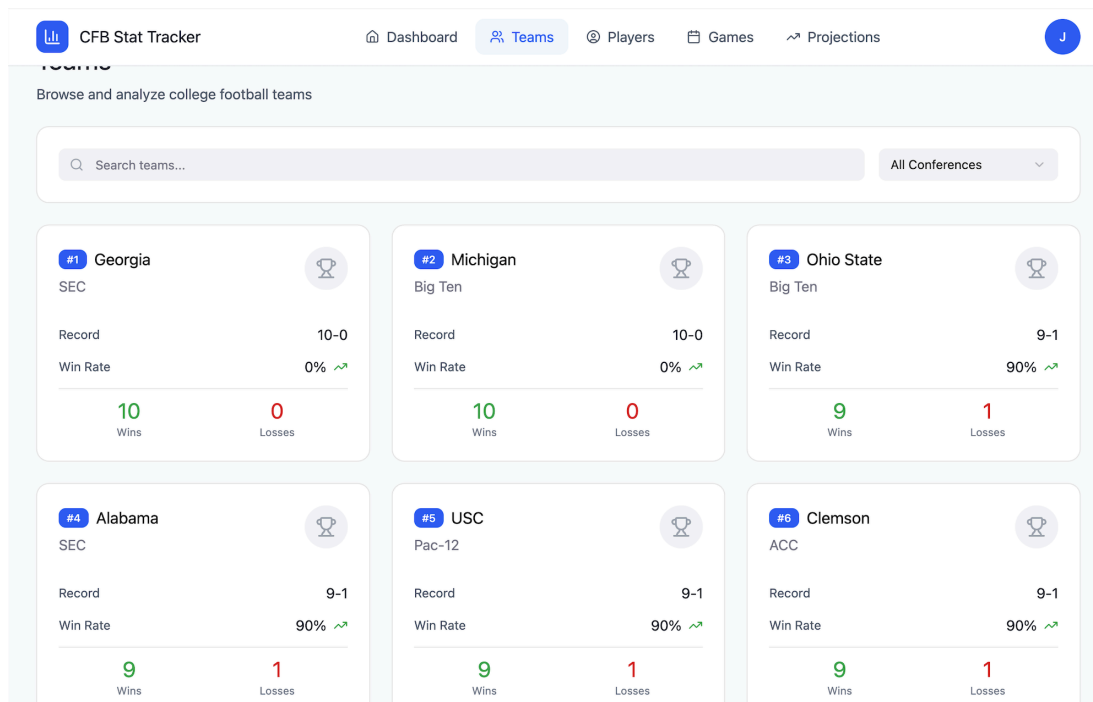
- Top KPI strip:
 - Cards showing metrics such as “Games Live Now”, “Completed Games Today”, “Top Performer (Passing Yards)”, “Top Performer (Rushing Yards)”.
 - Each card is clickable and deep-links into the relevant Teams, Players, or Games view.
 - Main chart area (center of the content panel):
 - Time-series charts displaying trends such as team scoring over the season, yardage leaders, or conference win distributions (driven by data from the Games and Stats tables).
 - Users can change the season/week using dropdowns in the left filter panel; charts update dynamically.
 - Live scoreboard strip (near bottom of dashboard):
 - Compact list of ongoing or recently finished games with current score, quarter, and clock.
 - Clicking a game row navigates to the Games View detail for that game.
- Guest users see the same layout but with Top 25 teams only and a “Login to view all teams and live projections” callout.



Teams View:

The Teams View focuses on browsing and comparing NCAA teams.

- Teams list/grid:
 - Each team is represented as a card showing: team name, conference, logo, basic record or key stat snippet.
 - Cards are color-tagged by conference for quick scanning.
- Filters and search (left panel):
 - Search by team name.
 - Filter by conference, season, or ranking tier (e.g., “Top 25 only”).
- Team detail panel (on select):
 - When a team card is selected, the right-hand main area shows:
 - Team logo and name.
 - Recent games (from the Games table) and mini charts (e.g., points scored per game across last N weeks).
 - Links to “View Players” (opens Players View filtered by this team) and “View Games” (Games View filtered by this team).

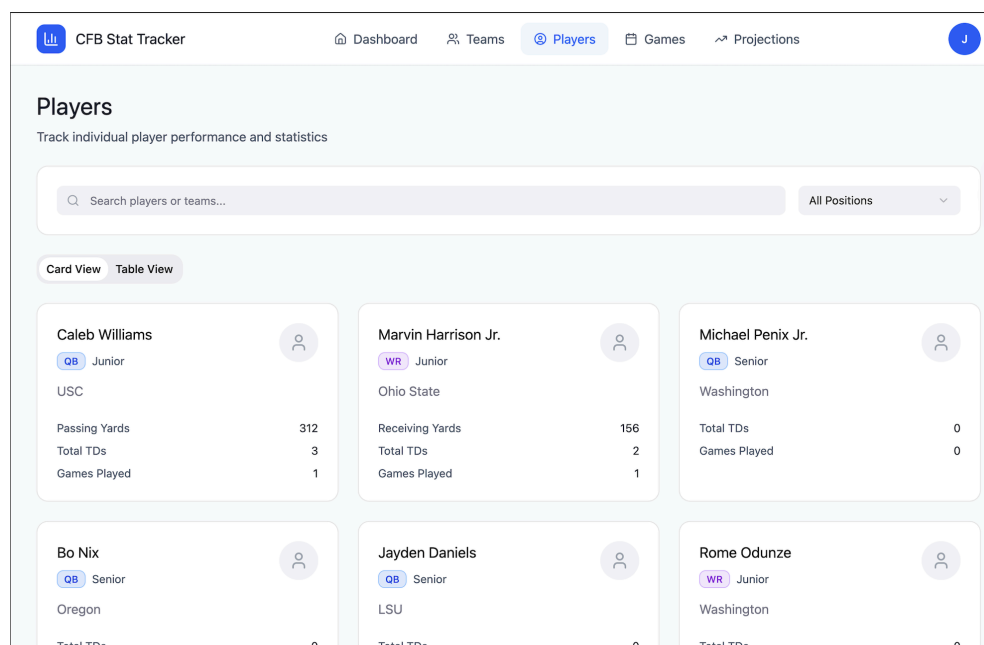


Players View:

The Players View is optimized for drill-down into individual performance and projections.

- Player table:

- Columns such as Player Name, Team, Position, Key Season Stats (e.g., passing_yards, rushing_yards, TDs); data is derived from Players and Stats tables.
 - Column sorting and text search (e.g., by name or position).
 - Filters:
 - Position filters (QB / RB / WR / etc.).
 - Team filter (synced with Teams View selection if navigated from there).
 - Player detail panel:
 - On row click, shows:
 - Season stat breakdown charts (e.g., bar chart of yards by game).
 - Recent games list with per-game stat lines.
 - For authenticated users, a Projections widget showing projected yards/TDs and confidence for upcoming games (reading from Projections table).
- Guest users may see projections blurred or locked with a tooltip explaining they require login.



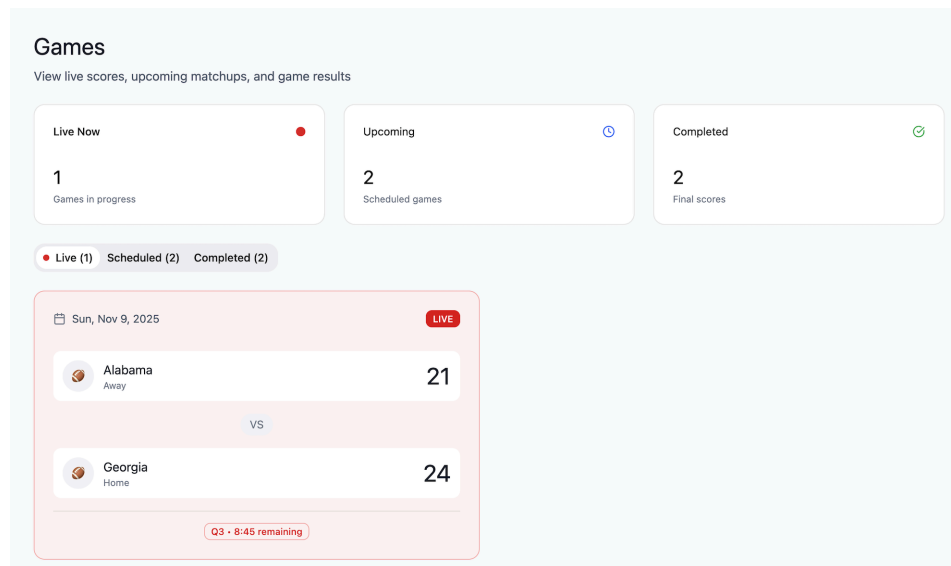
Games View:

The Games View presents game schedules, live scores, and game-level summaries.

- Game list / scoreboard layout:
 - Rows for each game including date/time, home team, away team, score (or scheduled), status.
 - Filters for season, week, and optionally conference.
- Game detail view:
 - When a game is selected:
 - Header with date, venue (if available), and final or current score.
 - Team summaries (home vs. away) with highlighting of winner.
 - Key statistics text/summary area populated from the key_statistics field in the Games table.

- Tabbed or expandable sections for box score and player stats, which reuse the Stats table data.

During live games, a subtle refresh indicator or “Updated X seconds ago” label appears, reflecting the ingestion/caching interval.



Projections View:

The Projections View is an advanced analytics page, primarily for authenticated users, surfacing results from the performance projection module.

- Projection table:

- Columns: Player, Team, Next Opponent, Projected Yards, Projected TDs, Confidence (mapped from the Projections table: projected_yards, projected_tds, confidence, joined to Players and Games).

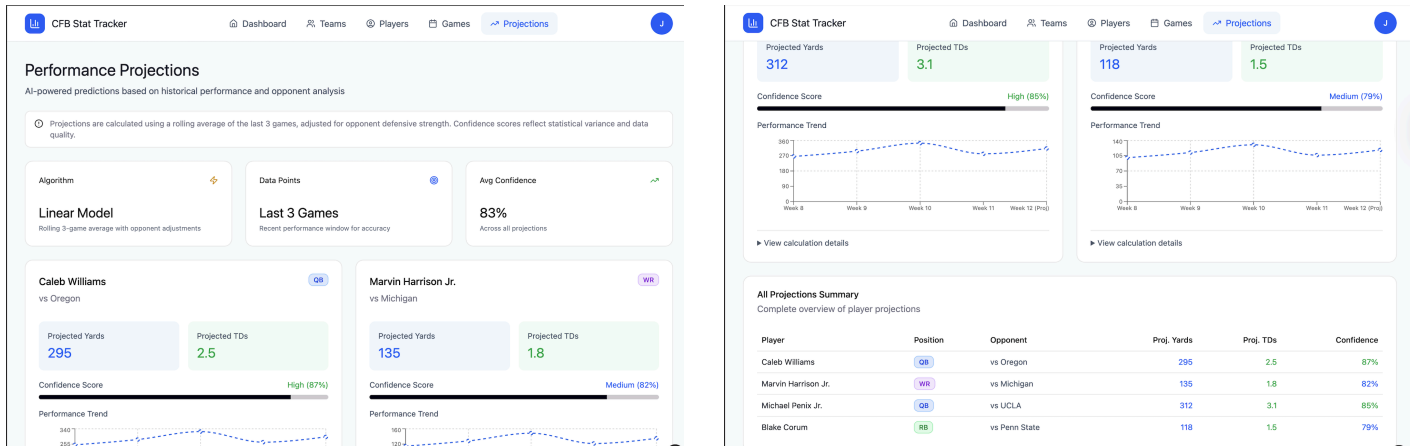
- Sort by projected yards, TDs, or confidence.

- Projection charts:

- Visualizations of projected vs. actual performance trends over recent games.

- Filters for season, week range, team, position.

Guest users either see this view disabled with a “Login to unlock projections” overlay or see a limited subset (e.g., only Top 10 projected players).



Difficult Algorithms / Core Logic

1) Performance Projection Module

Goal: estimate future player performance using recent historical stats.

Inputs:

- Last N games for the player
- Opponent defensive strength (if API provides)
- Player position (QB/RB/WR)

Algorithm (simple linear model):

1. Compute rolling averages:
 - $\text{avg_yards_3} = (\text{yards_game1} + \text{yards_game2} + \text{yards_game3}) / 3$
2. Optionally adjust for opponent:
 - $\text{adj} = \text{defensive_factor}(\text{opponent_team})$
3. Predicted yards:
 - $\text{pred} = w1 * \text{avg_yards_3} + w2 * \text{adj} + b$
4. Confidence:
 - $\text{confidence} = 1 - \text{variance}(\text{last_N_games})$

2) Data Ingestion with Idempotency

When the script runs every minute, we must not duplicate rows. So:

- Use INSERT ... ON DUPLICATE KEY UPDATE (MySQL)
- Or query if gameid already exists

- This ensures re-runs don't break the database

3) Live Refresh (Backend)

For live games, backend can:

- cache API responses for 30–60 seconds
- serve cached data to multiple users (reduces API calls)

Testing

I. Authentication and User Management (12 hours)

Test Case 1 – Black Box Testing – Google SSO login for new users

Input:

User clicks “Sign in with Google” and completes Google authentication for the first time.

Expected Output:

A new account is created in the database, the user is assigned the “user” role, and they are redirected into the application as a logged-in user.

Test Case 2 – Black Box Testing – Google SSO login for returning users

Input:

User clicks “Sign in with Google” using an email already registered in the system.

Expected Output:

User is authenticated and logged in with their existing role and account information (no duplicate account created).

Test Case 3 – Black Box Testing – Guest login behavior

Input:

User selects “Continue as Guest” instead of signing in with Google.

Expected Output:

A temporary guest session is created, the user is assigned the “guest” role, and the application loads with guest-level access.

Test Case 4 – Black Box Testing – Role-based access restrictions for guest users

Input:

Guest user attempts to open a feature restricted to authenticated users (e.g., viewing all seasons or all teams).

Expected Output:

Access is denied; user sees a message prompting them to sign in for full access.

Test Case 5 – Black Box Testing – Role-based access for logged-in users

Input:

Logged-in user attempts to open any full-access feature (e.g., full team list, multi-season stats).

Expected Output:

Access is granted with no restrictions.

Test Case 6 – Black Box Testing – Session timeout behavior

Input:

User stays inactive past the session timeout period.

Expected Output:

User is automatically logged out and must sign in again (or continue as guest).

Test Case 7 – Black Box Testing – Logout behavior

Input:

User clicks the “Logout” button.

Expected Output:

Active session is cleared, and the user is returned to the landing page where they can sign in again or continue as a guest.

II. System Architecture and Database Design (16 hours)

Test Case 1 – Black Box Testing – Storing user profiles in MySQL

Input:

Insert a new user record with email and role into the Users table.

Expected Output:

A new row is created with a unique userid, the correct email and role values, and a non-null created_at timestamp.

Test Case 2 – Black Box Testing – Enforcing team–player relationship

Input:

Attempt to insert a player with a teamid that does not exist in the Teams table.

Expected Output:

The insert is rejected and an error is returned, ensuring that every player must reference a valid team.

Test Case 3 – Black Box Testing – Retrieving players by team

Input:

Query all players belonging to a given teamid from the Players table.

Expected Output:

Only players whose teamid matches the requested teamid are returned.

Test Case 4 – Black Box Testing – Storing game results

Input:

Insert a game record into the Games table with specific values for date, home_team, away_team, scores, and key statistics.

Expected Output:

A new game row is created with a unique gameid and all fields stored correctly and retrievable without data loss.

Test Case 5 – Black Box Testing – Linking stats to players and games

Input:

Insert a stat record into the Stats table with valid playerid, gameid, season, type, and value.

Expected Output:

The stat row is created successfully and can be retrieved by joining Stats with Players and Games on playerid and gameid.

Test Case 6 – Black Box Testing – Prepared statements against SQL injection

Input:

Execute a parameterized query (prepared statement) with a malicious input string in a filter field (e.g., email or team name containing SQL keywords).

Expected Output:

The database returns only correctly filtered results or an empty set, and no SQL injection occurs (no unintended data modification or query execution).

Test Case 7 – Black Box Testing – Input validation on numeric/stat fields

Input:

Attempt to insert a stat record with an invalid value (e.g., a negative number for yards or a non-numeric string).

Expected Output:

The insert is rejected or an error is returned due to failed input validation, and no invalid stat record is stored.

Test Case 8 – Black Box Testing – SSL-encrypted database connection

Input:

Configure the application to connect without SSL and attempt to open a connection to the MySQL database.

Expected Output:

The connection attempt fails, and the system reports that SSL is required for the database connection.

III. Data Ingestion and API Integration (10 hours)

- Integrate with third-party sports data APIs to fetch live NCAA football stats and historical records.
- Schedule regular syncing and updates (every minute for live games).
- Parse, validate, and store incoming API data to the database.
- Implement robust error handling and logging for data integrity.

Test Case 1 – Black Box Testing – Valid API Data Ingestion

Input:

Trigger the ingestion process with a valid API response containing complete NCAA football game data.

Expected Output:

Game data is parsed successfully, validated, and inserted into the database. No errors appear in logs.

Test Case 2 – Component Test – Malformed API JSON Response

Input:

Provide an API response containing malformed or incomplete JSON, such as truncated text or invalid structure.

Expected Output:

The system rejects the response, logs a parsing error, and does not insert any records into the database.

Test Case 3 – White Box Testing – Missing Required Fields

Input:

Provide an API response where required fields (e.g., gameId or team statistics) are missing.

Expected Output:

Validation fails along the “missing field” branch. No database write occurs, and the log captures which field was missing.

Test Case 4 – Integration Testing – Scheduled Ingestion Execution

Input:

Allow the scheduler to run ingestion every minute for five consecutive cycles.

Expected Output:

Five ingestion cycles execute on time without overlap. The database reflects updates from each cycle, and logs record successful operations.

Test Case 5 – Stress Testing – High-Volume Game Load

Input:

Provide an API response containing 200–500 simultaneous live game records.

Expected Output:

All records are processed before the next scheduled ingestion. The system remains stable, no crashes occur, and the database stores all valid entries.

Test Case 6 – Error Handling Test – API Timeout or Service Unavailable

Input:

Simulate an API timeout, a slow response, or an HTTP 503 Service Unavailable response.

Expected Output:

The ingestion process times out or aborts gracefully. An error is logged, the database is not updated with partial data, and the system remains functional for the next cycle.

Test Case 7 – Functional Testing – Duplicate Game Updates

Input:

Provide identical API data across multiple ingestion cycles.

Expected Output:

The system detects duplicate data and avoids creating redundant database entries. Logs show either minimal changes or “no update required.”

Test Case 8 – Component Failure Testing – Database Write Failure

Input:

Simulate a database outage, connection loss, or write error during ingestion.

Expected Output:

The insertion attempt fails cleanly, the transaction is rolled back, an error is logged with the database error code, and no partial records appear in the database.

IV. Interactive Dashboard and User Interface

Test case 1 - Black Box Testing - Responsive Design

Input:

- User opens the College Football Stat Tracker in a desktop browser as a guest.
- User logs in (Google SSO/Auth0) and is redirected back to the Dashboard.
- User uses the left control panel to change global filters (season, week, conference, team).

Expected Output:

- The two-panel layout renders correctly:
 - Left ~1/5: vertical control panel with global filters and navigation shortcuts.
 - Right ~4/5: main content area with KPI cards, charts, and live scoreboard.
- For guests:
 - A banner indicates data is delayed and limited to Top 25 teams.
 - Advanced analytics/projections sections appear visually locked/disabled.
- For logged-in users:
 - Banner changes to “Live” data access with all teams available.
 - Projections and advanced analytics widgets become enabled.
- When filters are changed, all visible charts/tables/scoreboards update to reflect the new season/week/team without a full page reload, and the layout does not break (no overlapping or clipped elements).

Test case 2 - Black Box Testing - Sorting Functionality

Input:

- User navigates to the Players View from the top navigation bar.
- User selects filters in the left panel (e.g., Position = QB, Season = current).
- In the player table, user clicks on column headers to sort by:
 - Passing yards.
 - Touchdowns.
 - Confidence (in the projections column, if authenticated).

Expected Output:

- Player table initially shows players matching the selected filters (team/position/season).
- Clicking a sortable column header sorts the table by that column in descending order on first click and ascending order on second click.
- Sorting does not clear active filters (position/team) or pagination.
- For authenticated users:
 - Sorting by projected yards/TDs/confidence reorders rows based on data from the Projections table.
- For guests:

- If projections are partially locked/blurred, sorting still works on visible columns without causing errors or blank rows.
- No incorrect mixes of teams/positions occur after sorting (filters remain honored).

Test case 3 - Black Box Testing - Mobile Compatibility

Input:

- User opens the web app on a mobile device OR gradually resizes a desktop browser from full width down to a narrow/mobile width.
- User navigates between Dashboard, Teams, Players, Games, and Projections.
- User scrolls and interacts with tables (e.g., Players View) and charts on a narrow screen.

Expected Output:

- At tablet/mobile widths:
 - Top navigation collapses into a hamburger menu with access to Dashboard, Teams, Players, Games, Projections, and Login/Logout.
 - The left control panel collapses into a slide-out or stacked filter section above/below the main content.
- Charts and tables reflow so that:
 - Critical information (team name, player name, key stats) remains readable without horizontal scrolling when possible.
 - Less important columns hide or become horizontally scrollable as needed.
- Live scoreboard and key filters remain reachable without overlapping text or controls.
- All buttons, links, and interactive elements are finger-tap friendly (no tiny or overlapping hit targets).
- No layout breaks (e.g., cut-off charts, overlapping text, or unusable filters) occur when switching views or rotating the device.

Test Case 4 - Stress Testing - Large Schedule / Many Games

Input:

- Logged-in user sets filters to view an entire busy week of games:
 - Season = active season.
 - Week = a week with many scheduled and completed games.
- User navigates to Games View and:
 - Requests to see all games in that week (no conference/team filter).
 - Opens several game detail panels in succession.
 - Jumps from Games View to Teams View and back, keeping the same season/week context.

Expected Output:

- Games View successfully lists all games for the selected week (potentially dozens of games) with correct date/time, home/away teams, scores, and status.
- Opening any game shows:
 - Header with correct teams, date, and score.
 - Key statistics summary populated from the Games table.
 - Box score/player stats sections populated from the Stats table.
- Navigating between multiple game details remains responsive; no browser crashes or unhandled JavaScript errors occur.
- Schedule/game listing updates remain within a reasonable time frame (e.g., under a few seconds) even at peak weeks with many games.
- The UI remains interactive during loading (e.g., shows spinners/placeholders instead of freezing).

Test Case 5 - Stress Testing - Heavy Data Rendering

Input:

- Logged-in user performs high-volume browsing actions:
 - In Players View:
 - Removes most filters (e.g., all positions, all teams in FBS for a season).
 - Scrolls through a large table containing potentially thousands of players.
 - In Teams View:
 - Views all teams at once and quickly switches conferences/season filters.
 - In Projections View:
 - Requests projections for a large set of players (e.g., all QBs in the league).
- User rapidly scrolls, sorts columns, and opens multiple player/team detail panels in one session.

Expected Output:

- Large tables (Players, Teams, Projections) load and render without the browser crashing or freezing.
- Virtualization or pagination (if implemented) ensures scrolling remains smooth and responsive.
- Sorting and filtering on large datasets complete within acceptable times and do not lock the UI.
- No visual glitches (blank rows, duplicate rows, misaligned headers) appear when scrolling quickly or switching views.
- The user can continue to interact with the interface (filters, navigation, detail panels) without performance degradation, and no script errors appear in the console.

V. Performance Projection Module (16 hours)

- Implement statistical models (e.g., linear regression, moving averages) using Python or built-in JavaScript libraries to generate projections for upcoming games and players.
 - Visualize predicted outcomes in the dashboard.
 - Provide metrics for model accuracy and display confidence intervals.
-

VI. Multi-threading and Networking (8 hours)

- Backend uses asynchronous data fetching and multi-threaded processing for live stats updates.
 - Network requests are managed via API endpoints for robustness and speed.
-

VII. User Login Functionality (4 hours)

Test Case 1 – Black Box Testing – Real-time delay for guest users

- Input:
 - User attempts to use the application as a guest user
- Expected Output:
 - The user will experience a real-time delay of at least 10 minutes

Test Case 2 – Black Box Testing – Real-time delay for account users

- Input:
 - User attempts to use the application while logged into their account
- Expected Output:
 - The user will not experience any form of real-time delay

Test Case 3 – Black Box Testing – Season stat availability for guest users

- Input:
 - User attempts to retrieve season stat information as a guest user
- Expected Output:
 - The user will only be able to access stats from the most recent season

Test Case 4 – Black Box Testing – Season stat availability for account users

- Input:
 - User attempts to retrieve season stat information while logged into their account
- Expected Output:
 - The user will be able to retrieve stats from any season that they like, not just the most recent

Test Case 5 – Black Box Testing – Selecting teams to retrieve stats from as a guest user

- Input:
 - User is logged in as a guest and clicks on the drop-down menu to browse the selection of teams they can retrieve stats from
- Expect Output:
 - The user is given the option to browse the stats of only the select 25 teams available to guests

Test Case 6 – Black Box Testing – Selecting teams to retrieve stats from as an account user

- Input:
 - User is logged in as a guest and clicks on the drop-down menu to browse the selection of teams they can retrieve stats from
- Expected Output:
 - The user is given the option to browse the stats of all the teams available, and is also given advanced stats option that the guests do not have access to

Test Case 7 – Black Box Testing – Attempting to access projections data as a guest user

- Input:
 - User attempts to access the projections data as a guest
- Expected Output:
 - The user is informed that the projections data is not available for guest users, and that they need to create an account to view it

Test case 8 – Black Box Testing – Attempting to access projections data as an account user

- Input:
 - User attempts to access the projections data while logged into their account
- Expected Output:
 - The projections data will be displayed to the user in table format so that it is easy for the user to interpret

Test case 9 – Black Box Testing – Attempting to access odds data as a guest user

- Input:
 - User attempts to access the odds data as a guest
- Expected Output:
 - The user is informed that the odds data is not available for guest users, and that they need to create an account to view it

Test Case 10 – Black Box Testing – Attempting to access the odds data as an account user

- Input:
 - User attempts to access the odds data while logged into their account
 - Expected Output:
 - The odds data will be displayed to the user in table format so that it is easy for the user to interpret
-

VIII. Testing, Deployment, and Maintenance (16 hours)

- Version control managed with GitHub (CI/CD optional for full points).
- Implement unit, integration, and system-level tests (using Jest, PyTest, etc.).
- Deploy on a cloud platform (AWS/GCP/Azure); ensure routine backups.

Test Case 1 – Integration Testing – GitHub version control workflow

Input:

- Developer creates a feature branch from main.
- Commits changes to backend + frontend.
- Opens a Pull Request (PR) into main on GitHub.

Expected Output:

- All commits appear in the PR with proper diff.
- PR shows no merge conflicts with main.
- Code review and status checks (lint/tests) are visible on the PR.
- Merging the PR updates main without losing history or overwriting others' changes.

Test Case 2 – Integration Testing – Automated test suite on CI

Input:

- A PR is opened or updated on GitHub.

- CI pipeline (e.g., GitHub Actions) is configured to run:
 - Frontend tests (e.g., Jest/Vitest).
 - Backend tests (e.g., PyTest/JUnit).

Expected Output:

- CI pipeline automatically triggers on PR updates.
- All unit and integration tests run and report pass/fail status.
- If any tests fail, CI marks the PR as failing and blocks merge (depending on repo settings).
- Logs clearly show which tests failed and why.

Test Case 3 – System Testing – Staging deployment to cloud

Input:

- Code is merged into a designated staging or develop branch.
- CI/CD pipeline deploys the app to a staging environment on a cloud platform (AWS/GCP/Azure/Render/etc.).

Expected Output:

- Build and deployment steps complete successfully (no build errors).
- Staging URL becomes reachable.
- Application loads with the expected configuration (e.g., using staging DB, staging API keys).
- Sanity checks (login, dashboard load, basic navigation) all work in staging.

Test Case 4 – System Testing – Production deployment

Input:

- Code is merged into main (or tagged release).
- CI/CD pipeline deploys to production environment.

Expected Output:

- Deployment completes without errors.
- Production URL stays available during deployment or shows a short, controlled maintenance state.
- Application runs with production config (prod DB, proper Auth0 domain, etc.).
- Health check endpoint (if defined) returns success.

Test Case 5 – Reliability Testing – Rollback capability**Input:**

- Deploy a known “bad” version (simulated bug) to production or staging.
- Attempt to roll back using your deployment strategy (e.g., previous container image, previous release tag).

Expected Output:

- Rollback command/process completes successfully.
- Application returns to previous, stable version.
- No data loss from the rollback (schema unchanged, or backward compatible).
- Monitoring/alerts show recovery after rollback.

Test Case 6 – Maintenance Testing – Database backup and restore**Input:**

- Trigger a routine backup of the production database (automatic or manual).
- Simulate a data loss scenario in a test environment (e.g., drop tables or corrupt test DB).
- Restore from the most recent backup into a test DB instance.

Expected Output:

- Backup file is successfully created and stored in the configured backup location (e.g., S3).
- Restore completes without errors.
- Restored database contains all expected tables and data up to the backup timestamp.
- Application can connect to the restored DB and function normally (in the test environment).

Test Case 7 – Monitoring & Logging – Error and performance visibility

Input:

- Trigger a controlled backend error (e.g., invalid API request) and a slow query or request.
- Observe logging and monitoring tools (CloudWatch, Stackdriver, APM, etc.).

Expected Output:

- Error is logged with stack trace, timestamp, and request context.
- Slow request shows up in performance metrics (e.g., high latency trace, slow query log).
- Alerts (if configured) fire for threshold-breaching errors or response times.
- Logs/metrics can be used to pinpoint and debug the issue.

IX. Data Structures (4 hours)

- Store team/player/game info in normalized database tables.
- Use in-memory data structures (e.g., HashMaps for quick lookups, Queues for update scheduling) within backend services.

Test Case 1 – Black Box Testing – Normalized team/player/game tables

Input:

- Insert multiple teams into Teams, multiple players into Players (with teamid), and multiple games into Games.
- Insert stats into Stats referencing playerid and gameid.

Expected Output:

- No duplicated team or player rows for the same real-world entity (e.g., no two rows with same team_name + conference unless intended).
- All Players.teamid values reference existing rows in Teams.
- All Stats.playerid and Stats.gameid values reference existing Players and Games.
- Queries joining Teams–Players–Games–Stats return consistent, non-duplicated logical entities.

Test Case 2 – Integration Testing – HashMap / dictionary lookups for teams and players

Input:

- Backend loads teams and players into in-memory maps:
 - teamIdToTeam (HashMap by teamid)
 - playerIdToPlayer (HashMap by playerid)

- Execute multiple lookups by ID while serving API requests.

Expected Output:

- Lookup by ID returns the correct object in $O(1)$ -ish time (no noticeable delay).
- Non-existent IDs return null/empty or handled error, not a random object.
- Updates to the DB (e.g., new teams/players) are reflected in the in-memory map after refresh/reload.

Test Case 3 – White Box Testing – Queues for update scheduling

Input:

- Enqueue multiple update tasks (e.g., “fetch live stats for Game A”, “Game B”, etc.) into a Queue structure.
- Process tasks in order.

Expected Output:

- Tasks are dequeued and executed in the expected order (FIFO), unless a different scheduling rule is explicitly implemented.
- No tasks are lost or processed more than once under normal operation.
- On application restart, either:
 - The queue is reconstructed from persisted state, or
 - Tasks are safely re-queued from a known schedule source without duplication.

Test Case 4 – Performance Testing – Scale of in-memory caches

Input:

- Load a large number of entries into in-memory data structures (e.g., all FBS players for a season).
- Perform repeated lookups and updates.

Expected Output:

- Memory usage remains within acceptable limits for the expected data size.
- Lookup time remains fast (no significant degradation as the cache grows).
- Eviction or refresh strategy (if implemented) correctly removes/stales old entries without breaking lookups.

X. Skills and Tools

- Web frameworks (React/Vue.js/Angular), RESTful API integration, Python/R for data modeling, MySQL for backend storage.
- Version control: Git/GitHub.
- Data visualization: D3.js, Chart.js, or equivalent.

Test Case 1 – Environment Verification – Frontend framework setup

Input:

- Run npm install / yarn in the frontend project.
- Run the dev server (npm run dev / npm start).

Expected Output:

- Dependencies install without critical errors.
- Development server starts successfully.

- App loads in the browser at the dev URL, showing the base layout (nav + dashboard shell).
- Hot reloading works: small code changes reflect in the browser without a full restart.

Test Case 2 – Integration Testing – RESTful API communication

Input:

- Frontend calls backend endpoints (e.g., /api/teams, /api/players, /api/games/{id}) using fetch/axios.
- Network is functioning normally.

Expected Output:

- Requests reach the backend and return JSON in the expected format (status 2xx).
- Frontend parses JSON and renders the data in tables/charts.
- Error handling path works: if the backend returns 4xx/5xx, the UI shows a friendly message instead of crashing.

Test Case 3 – Tooling Testing – Python/R modeling environment

Input:

- Run the data modeling scripts (Python or R) that compute projections/analytics.
- Scripts connect to the database or read from prepared CSV/JSON files.

Expected Output:

- Required libraries (e.g., pandas, scikit-learn, R packages) are installed and import successfully.
- Scripts run end-to-end without exceptions.

- Output files (e.g., CSVs with projections) or DB writes are generated in the correct format for the web app to consume.

Test Case 4 – Integration Testing – Database connectivity from backend

Input:

- Backend starts up with configured MySQL connection string.
- Executes test queries on startup (e.g., simple SELECT 1, or SELECT COUNT(*) FROM Teams).

Expected Output:

- Database connection is established successfully.
- Health check endpoint reports DB as “healthy/connected”.
- If the DB is unavailable, backend logs a clear error and (optionally) exposes a “degraded/unhealthy” status instead of silently failing.

Test Case 5 – Visualization Testing – Charts and graphs rendering

Input:

- Navigate to dashboards that use D3.js / Chart.js (or equivalent) for:
 - Team performance over time.
 - Player stats distribution.
 - Win probability or projection charts.

Expected Output:

- Charts render without JavaScript errors.
- Axes labels, legends, and tooltips display correctly and match the underlying data.

- Interactions like hover, tooltip, or legend toggling behave as expected.
- For empty datasets (no stats found), charts show a sensible “No data” state instead of breaking the page.

Test Case 6 – Version Control Proficiency – Git/GitHub usage

Input:

- Developer clones the repo, creates a feature branch, commits changes with meaningful messages, and pushes to GitHub.
- Developer resolves a simple merge conflict between two branches.

Expected Output:

- Repository can be cloned and built from scratch by any team member.
- New branches and commits appear correctly on GitHub.
- Merge conflicts are resolved cleanly without losing code.
- main remains in a deployable state after merges.

Deployment

1. Set up the server

- Create a Linux machine (e.g., on AWS).
- Install Java, Node.js, Python, and MySQL.

2. Get the project onto the server

- Install Git.
- `git clone` the College Football Stat Tracker repo.

3. Create the database

- Start MySQL.
- Create the production database and user.
- Run your SQL schema file to create tables.

4. Add configuration / secrets

- Create a config file or `.env` with:
 - DB username/password
 - Sports API keys
 - Auth0 / Google SSO keys
 - Any other URLs or secrets.

5. Build and run the Java backend

- Run `mvn clean package`.
- Deploy the WAR to Tomcat **or** run the JAR directly.

- Check that /api/teams or another test endpoint works.

6. **Build the React frontend**

- Go to the frontend folder.
- Run npm install then npm run build.
- Configure the server to serve the build folder.

7. **Start the Python data scripts**

- Create a virtual environment and install requirements.
- Run your ingestion script once to test.
- Set it up with cron or systemd so it runs automatically.

8. **Final checks and go live**

- Test login, dashboard pages, and a few sample stats.
- Add HTTPS and point your domain name to the server.
- Tell users the final URL of the app.